

OPERATING SYSTEMS (CSE-335)

Full Question Paper Transcription with Full Solutions

Source file: /home/runner/work/KD/KD/OS_Term_Qsn_(21-18).pdf

This document is arranged exactly as requested: all **Section A** questions and solutions first, then all **Section B** questions and solutions. Where the original scan is partially unclear, the question text is reconstructed from the readable portions and the solution clearly states the working assumptions.

Included diagrams in this PDF

1. Process-state diagram
2. Directory-structure sketches
3. Deadlock/resource-allocation sketch
4. Inverted page table sketch
5. Paging address-translation sketch
6. Page-fault handling flow sketch
7. Semaphore-ordering sketch

SECTION A - FULL QUESTIONS AND FULL SOLUTIONS

2024 Exam - Section A

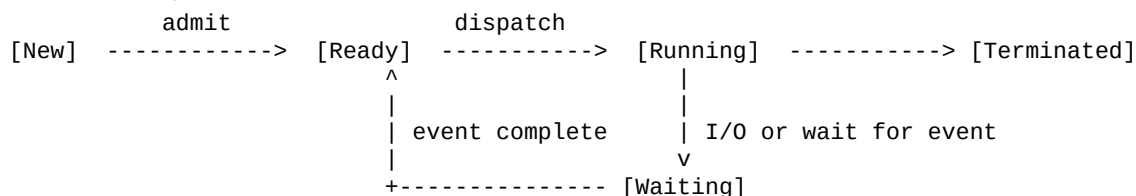
Q1(a) Process states and transitions

Question: Create a diagram illustrating different process states and transitions. Use your diagram to show the sequence of states a process may go through.

Solution:

- The standard process-state model contains five main states: **New**, **Ready**, **Running**, **Waiting/Blocked**, and **Terminated**.
- A process starts in the **New** state when it is created by the system.
- After admission, it moves to **Ready** and waits in the ready queue for CPU allocation.
- The scheduler dispatches it to **Running** when the CPU becomes available.
- If the process requests I/O or waits for an event, it enters **Waiting**.
- When the I/O or event completes, the process returns to **Ready**.
- If the time quantum expires or a higher-priority process arrives, the running process may be preempted back to **Ready**.
- When execution finishes, the process moves to **Terminated**.

Process-state diagram



A typical sequence:

New -> Ready -> Running -> Waiting -> Ready -> Running -> Terminated

Q1(b) Directory structures

Question: Explain Tree-Structured, Acyclic-Graph, and General-Graph directory structures with necessary diagrams.

Solution:

- **Tree-structured directory:** every file or subdirectory has exactly one parent. It is simple to manage and traverse.
- **Acyclic-graph directory:** allows sharing through links, but cycles are not allowed. This supports controlled sharing.
- **General-graph directory:** permits cycles. It is the most flexible but requires cycle detection during traversal and deletion.
- Tree structure is best when simplicity is the goal; acyclic graph is useful when sharing is needed without infinite loops; general graph offers maximum flexibility but highest management cost.

Directory-structure sketches

Tree structure:

```

root
+- home
| +- student
|   +- notes.txt
+- usr
  +- bin
  
```

Acyclic graph:

```

root
+- courseA
| +- shared.pdf ---+
+- courseB         |
  +- link          -----+
  
```

General graph:

```

root
+- A
|   +- B
|       +- C
|           +- link back to A (cycle)

```

Q2(a) IPC mechanisms

Question: Compare and contrast different interprocess communication (IPC) mechanisms.

Solution:

- **Shared memory:** fastest after setup because processes access the same memory region directly; however, semaphores/mutexes are required for synchronization.
- **Message passing:** communication occurs via send/receive operations. It is easier to protect and fits distributed systems naturally, but it has higher overhead than shared memory.
- **Pipes/FIFOs:** simple unidirectional or bidirectional byte-stream communication, mainly used between related or cooperating processes.
- **Sockets:** flexible IPC for local or remote communication. They are essential for client-server and networked applications.
- **Signals:** useful for short notifications and asynchronous events, but not suitable for transferring large data.
- Therefore, shared memory is best for speed, message passing for modularity and safety, sockets for network communication, and signals for lightweight notification.

Mechanism	Main strength	Main limitation
Shared memory	Very fast data exchange	Needs explicit synchronization
Message passing	Simple isolation and protection	Higher kernel overhead
Pipes/FIFOs	Easy to use	Limited flexibility
Sockets	Works locally and across network	More setup overhead
Signals	Fast event notification	Very small payload

Q2(b) CPU scheduling criteria and Round Robin time quantum

Question: What criteria are used to measure the performance of a CPU scheduling algorithm? Explain the effect of increasing and decreasing the time quantum for the Round Robin scheduling algorithm.

Solution:

- **CPU utilization:** percentage of time the CPU remains busy. Higher is better.
- **Throughput:** number of processes completed per unit time. Higher is better.
- **Turnaround time:** completion time minus arrival time. Lower is better.
- **Waiting time:** total time spent in the ready queue. Lower is better.
- **Response time:** time from submission until first response. Lower is better.
- **Fairness:** each process should get a reasonable chance to execute; starvation should be avoided.
- If the Round Robin time quantum becomes **very large**, the algorithm behaves like FCFS, so response time for interactive jobs becomes poor.
- If the time quantum becomes **very small**, the system spends too much time in context switching, reducing useful CPU work.
- A **moderate** time quantum gives a balance between responsiveness and context-switch overhead.

Q2(c) Priority-preemptive and SRTF scheduling

Question: Consider the following processes: P1(AT=0,BT=7,Priority=3), P2(AT=2,BT=4,Priority=3), P3(AT=4,BT=1,Priority=4 - lowest), P4(AT=3,BT=4,Priority=1 - highest), P5(AT=3,BT=2,Priority=1 - highest).

(i) Draw the Gantt charts for Priority-Preemptive and SRTF. (ii) Compute average turnaround time, waiting

time, and response time for the two algorithms.

Solution:

- Assume smaller priority number means higher priority.
- For **Priority-Preemptive**, P1 starts first, then P4 and P5 preempt because they have higher priority.
- For **SRTF**, the process with the shortest remaining time is always selected.
- The computed metrics are shown below.

Algorithm	Average Turnaround	Average Waiting	Average Response
Priority-Preemptive	10.4	6.8	5.6
SRTF	7.8	4.2	1.4

Gantt chart - Priority-preemptive

```

0       3       7       9       13       17       18
| P1 | | P4 | | P5 | | P1 | | P2 | | P3 |

```

Gantt chart - SRTF

```

0   2   3   5   6   9   13   18
|P1|P2|P5|P3| P2 |  P4  |  P1  |

```

Q3(a) Deadlock scenario using a resource-allocation graph

Question: Given a resource allocation graph with processes P1, P2, P3 and resources R1, R2, R3, draw a scenario that leads to deadlock. Explain why deadlock occurs and how it can be detected.

Solution:

- Let P1 hold R1 and request R2, P2 hold R2 and request R3, and P3 hold R3 and request R1.
- Then each process is waiting for a resource currently held by another process.
- This creates a circular wait, so none of the processes can proceed.
- Deadlock can be detected by converting the graph to a wait-for graph and checking for a cycle when each resource has a single instance.
- For multiple instances, a detection algorithm based on allocation/request matrices is used.

Deadlock sketch

R1 assigned to P1, P1 requests R2
 R2 assigned to P2, P2 requests R3
 R3 assigned to P3, P3 requests R1

Cycle:

P1 -> R2 -> P2 -> R3 -> P3 -> R1 -> P1

Q3(b) Tape-drive state classification

Question: A system has 12 tape drives. Maximum needs are P1=10, P2=4, P3=9. Current allocations at time t1 are P1=5, P2=2, P3=3. Determine whether the system is in a safe state, unsafe state, deadlocked state, or starvation condition. Justify the answer.

Solution:

- Total allocated tapes = 5 + 2 + 3 = 10, so available tapes = 12 - 10 = 2.
- Remaining needs are P1=5, P2=2, P3=6.
- With 2 available tapes, only P2 can finish. After P2 completes, it releases 2 tapes, so available becomes 4.
- Now P1 still needs 5 and P3 still needs 6. Neither can complete with only 4 available tapes.
- Therefore, there is no safe sequence from this state.
- **Conclusion:** the system is in an **unsafe state**. Unsafe does not always mean deadlocked immediately, but it means deadlock may occur later.

Q3(c) Resource-allocation graph questions

Question: Answer the following: (i) If P2 makes an additional request for R1 in the given RAG, can the system grant it safely? (ii) A cycle in the RAG is a necessary condition for deadlock. Is it always a sufficient condition? Explain briefly.

Solution:

- An additional request can be granted safely only if the resulting state remains safe and does not force a deadlock.
- So the correct method is: tentatively grant the request, test the state, and grant it only if a safe execution sequence still exists.
- A cycle is **always sufficient** for deadlock only when each resource type has a **single instance**.
- When a resource type has **multiple instances**, a cycle is necessary but may not be sufficient because another free instance may still allow progress.

Note: The scan contains only a partial figure for part (i), so the exact yes/no result depends on the hidden graph details. The decision rule above is the correct full solution method.

Q4(a) RAID concepts

Question: Fill in the table for RAID 0, RAID 1, RAID 4, and RAID 5. Also explain how RAID 4 differs from RAID 5 in parity placement, and in the best case how many disk failures RAID 1 can tolerate without data loss.

Solution:

- RAID 0 uses striping only, so it improves performance but provides no redundancy.
- RAID 1 uses mirroring, so it provides strong reliability but requires high storage overhead.
- RAID 4 uses block-level striping with a dedicated parity disk. This dedicated parity disk can become a write bottleneck.
- RAID 5 also uses block-level striping, but parity is distributed over all disks, reducing the dedicated-disk bottleneck.
- In the best case, RAID 1 can survive one disk failure from each mirror pair without losing data.

RAID level	Organization	Fault tolerance	Space overhead	Performance summary
RAID 0	Striping	None	Low	Very fast reads/writes
RAID 1	Mirroring	High	50% usable capacity	Fast reads, safe writes
RAID 4	Striping + dedicated parity	Can survive one disk failure	One parity disk	Good reads, parity bottleneck on w
RAID 5	Striping + distributed parity	Can survive one disk failure	One-disk equivalent parity	Balanced performance

Q4(b) Disk scheduling

Question: A disk has cylinders 0 to 2999. Pending requests in FIFO order are 450, 200, 900, 100, 500, 600. The head starts at cylinder 550. Find the total head movement for FCFS, C-LOOK, SSTF, C-SCAN, and LOOK.

Solution:

- **FCFS:** 550 -> 450 -> 200 -> 900 -> 100 -> 500 -> 600 gives total movement 2350 cylinders.
- **SSTF:** 550 -> 500 -> 450 -> 600 -> 900 -> 200 -> 100 gives total movement 1350 cylinders.
- **LOOK** (initial direction upward): 550 -> 600 -> 900 -> 500 -> 450 -> 200 -> 100 gives total movement 1150 cylinders.
- **C-LOOK** (upward): 550 -> 600 -> 900 -> 100 -> 200 -> 450 -> 500 gives total movement 1550 cylinders.
- **C-SCAN** (upward): 550 -> 600 -> 900 -> 2999 -> 0 -> 100 -> 200 -> 450 -> 500 gives total movement 5948 cylinders.

Algorithm	Total head movement
FCFS	2350
SSTF	1350

LOOK	1150
C-LOOK	1550
C-SCAN	5948

2021 Exam - Section A

Q1(a) Security problems in a time-shared environment

Question: In a multiprogramming and time-sharing environment, several users share the system simultaneously. (i) What are two security problems that can arise? (ii) Can we ensure the same degree of security in a time-shared machine as in a dedicated machine? Explain.

Solution:

- Two common security problems are **unauthorized access to another user's memory/data** and **improper privilege escalation or account misuse**.
- Another possible problem is leakage through shared devices, temporary files, or residual memory blocks.
- A dedicated machine has a smaller exposure surface because a single user controls the environment.
- A time-shared machine can be made highly secure by using isolation, authentication, access control, auditing, encryption, and sandboxing.
- However, in practice it is difficult to claim the same absolute security as a dedicated machine because multiple users and shared resources increase the attack surface.

Q1(b) Cached-data coherence

Question: Discuss, with examples, how the problem of maintaining coherence of cached data manifests itself in (i) single-processor systems, (ii) multiprocessor systems, and (iii) distributed systems.

Solution:

- **Single-processor system:** the CPU cache may hold stale data if a DMA controller or device updates main memory directly.
- **Multiprocessor system:** one core may modify a cached block while another core still keeps the old copy; coherence protocols such as MESI are needed.
- **Distributed system:** replicas of the same data at different machines can diverge, so consistency protocols, versioning, or invalidation are required.
- Thus, coherence becomes progressively harder as the system becomes more parallel and more geographically distributed.

Q1(c) Open-source operating systems and peer-to-peer applications

Question: Identify several advantages and disadvantages of open-source operating systems. List some applications appropriate for a peer-to-peer system.

Solution:

- **Advantages:** low cost, source transparency, customizability, broad community support, and no vendor lock-in.
- **Disadvantages:** support quality may vary, some hardware/software compatibility issues may appear, and enterprise integration may require extra effort.
- Peer-to-peer applications include file sharing, decentralized backup, content distribution, blockchain networks, collaborative sharing platforms, and direct communication systems.

Q2(a) Two modes of IPC

Question: What are the two modes of interprocess communication? What are the strengths and weaknesses of the two approaches?

Solution:

- The two major IPC modes are **shared memory** and **message passing**.
- Shared memory is fast and suitable for high-volume data exchange, but it requires careful synchronization.
- Message passing is easier to protect and design, and it works naturally in distributed systems, but it is slower due to system-call and copying overhead.

Q2(b) iOS and Android

Question: How are iOS and Android similar? How are they different? Explain why Java programs running on Android do not use the standard Java API and virtual machine.

Solution:

- Both are modern mobile operating systems supporting applications, multitasking, networking, sensors, touch interfaces, and security controls.
- iOS is tightly controlled by Apple and targets a smaller hardware range, while Android is more open and runs on a wide variety of devices from many manufacturers.
- Android applications historically ran on Dalvik and later ART, not on the standard desktop/server JVM.
- Android also uses its own framework libraries and SDK APIs, so standard Java SE APIs are not used as-is.

Q2(c) OS file-management activities

Question: What are the five major activities of an operating system with regard to file management?

Solution:

- Create and delete files.
- Create and delete directories.
- Provide primitives to manipulate files and directories.
- Map files onto secondary storage.
- Back up files and provide protection/recovery mechanisms.

Q2(d) Context switching, memory stall, and soft deadline

Question: Explain the concepts of context switching, memory stall, and soft deadline.

Solution:

- **Context switching:** saving the state of the currently running process/thread and loading another one so the CPU can switch execution.
- **Memory stall:** the CPU stops useful progress temporarily while waiting for data to arrive from memory, usually after a cache miss.
- **Soft deadline:** a timing requirement whose violation degrades quality or usefulness, but does not produce catastrophic failure.

Q3(a) CPU scheduling criteria

Question: What criteria are used to measure the performance of a CPU scheduling algorithm?

Solution:

- CPU utilization, throughput, turnaround time, waiting time, response time, and fairness/starvation avoidance are the main criteria.

Q3(b) Scheduling problem

Question: Consider the processes P1(0,7,3), P2(2,4,2), P3(4,1,4), P4(5,4,1), P5(3,2,1). (i) Draw the Gantt charts for Priority-Preemptive and SRTF. (ii) Compute the average turnaround time if the CPU is left idle for the first 2 units and then SRTF is used.

Solution:

- For the priority-preemptive policy, the highest-priority ready process is selected whenever a new process arrives.
- For SRTF, the process with the least remaining execution time is always selected.

- If the CPU stays idle for the first 2 time units, P1 is deliberately delayed and therefore its completion time increases.
- The average turnaround time for the requested idle-then-SRTF case is **8.2** time units.

Gantt chart - Priority-preemptive

```

0   2   3   5   9   12   17  18
|P1|P2|P5| P4 | P2 | P1 |P3|

```

Gantt chart - SRTF

```

0   2   3   5   6   9   13   18
|P1|P2|P5|P3| P2 | P4 | P1 |

```

Gantt chart - CPU idle for first 2 units, then SRTF

```

0       2   3   5   6   9   13       20
| IDLE |P2|P5|P3| P2 | P4 | P1 |

```

Q3(c) Effect of Round Robin quantum

Question: Explain the effect of increasing the time quantum to an arbitrarily large number and decreasing it to an arbitrarily small number for the Round Robin scheduling algorithm, with suitable example.

Solution:

- If the quantum becomes very large, Round Robin degenerates into FCFS because each process tends to finish before preemption.
- If the quantum becomes very small, the scheduler switches too often and context-switch overhead rises sharply.
- Hence, a medium-sized quantum is preferred so that interactive processes get a quick response without wasting too much CPU time on switching.

Q4(a) Deadlock and necessary conditions

Question: What is a deadlock? What are the necessary conditions for a deadlock to occur?

Solution:

- Deadlock is a condition in which a set of processes waits forever because each process is waiting for a resource/event that can be released only by another waiting process in the same set.
- The four necessary conditions are: **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**.

Q4(b) Resource-holding/request situation

Question: A system has three types of resources R1, R2, and R3. The scanned question lists four processes with current holdings and additional requests. Determine which, if any, are deadlocked in this state.

Solution:

- The correct solution method is to compare total instances, current allocations, outstanding requests, and then build the wait relation.
- A process is deadlocked only if it belongs to a circular waiting chain and the requested resources cannot be satisfied by any future release outside that chain.
- Because the scan contains internal numeric inconsistencies in the resource counts, an exact unique deadlock set cannot be recovered with complete confidence from the visible image alone.

Note: The resource-instance counts in the scan are partially corrupted by OCR, so the reliable answer is the detection method rather than a forced exact set.

Q4(c) Banker's algorithm state check

Question: Consider the given Allocation and Max snapshot. Using Banker's algorithm, determine whether each of the following states is unsafe. If a state is safe, show a completion order: (i) Available = (0,3,0,1), (ii) Available = (1,0,0,2).

Solution:

- Using the readable values from the scan, compute **Need = Max - Allocation** for each process.

- For **Available = (0,3,0,1)**, the system can complete some processes, but it gets stuck before all processes finish, so the state is **unsafe**.
- For **Available = (1,0,0,2)**, a safe sequence exists.
- One valid safe sequence for case (ii) is **P1 -> P2 -> P3 -> P4 -> P0** under the reconstructed table.

Note: This answer follows the reconstructed numeric table from the readable scan. If the original handwritten/printed digits differ, the safe sequence may change.

SECTION B - FULL QUESTIONS AND FULL SOLUTIONS

2024 Exam - Section B

Q4(a) MMU, fragmentation, and TLB

Question: Answer the following: (i) What is the purpose of the memory management unit (MMU) in main memory? (ii) Discuss the differences between external and internal fragmentation. (iii) How does the TLB affect effective memory access time?

Solution:

- The **MMU** translates logical/virtual addresses into physical addresses and enforces protection, relocation, and access control.
- **External fragmentation** occurs when enough free memory exists in total but it is split into noncontiguous holes.
- **Internal fragmentation** occurs when allocated memory blocks/pages contain unused space inside them.
- A **TLB** is a fast associative cache for page-table entries. On a TLB hit, address translation is completed much faster, which reduces the effective memory access time.

Q5(b) Inverted page table

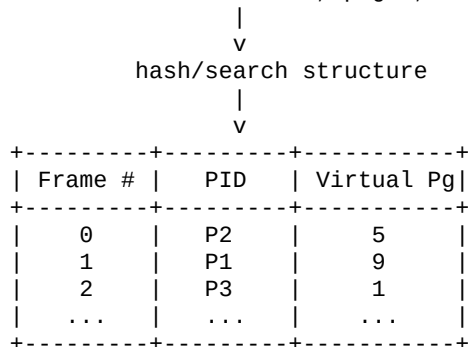
Question: With a neat sketch, explain the structure and working of an inverted page table in memory management.

Solution:

- A conventional page table has one entry per virtual page of each process. An inverted page table has one entry per **physical frame**.
- Each entry stores the process ID, virtual page number, and control bits for the page currently occupying that frame.
- During translation, the system searches the inverted table (often through hashing) to find the frame containing the requested virtual page.
- The main benefit is reduced page-table memory consumption for systems with large virtual address spaces.

Inverted page table sketch

Virtual address = <PID, page#, offset>



When a matching <PID, page#> entry is found, the frame number is combined with the offset.

Q5(c) Segment table address translation

Question: Consider the segment table: S0(base=520, limit=500), S1(base=350, limit=150), S2(base=100, limit=200), S3(base=1800, limit=600), S4(base=1050, limit=650). Find the physical addresses for (i) (2,50), (ii) (4,700), (iii) (0,400), (iv) (1,100), (v) (3,500).

Solution:

- Rule: a logical address (segment, offset) is valid only if **offset < limit**. If valid, **physical address = base + offset**.
- (i) (2,50): 50 < 200, so valid. Physical address = 100 + 50 = **150**.
- (ii) (4,700): 700 is not less than 650, so the reference is **invalid**.

- (iii) (0,400): $400 < 500$, so valid. Physical address = $520 + 400 = \mathbf{920}$.
- (iv) (1,100): $100 < 150$, so valid. Physical address = $350 + 100 = \mathbf{450}$.
- (v) (3,500): $500 < 600$, so valid. Physical address = $1800 + 500 = \mathbf{2300}$.

Logical address	Result
(2,50)	150
(4,700)	Invalid reference
(0,400)	920
(1,100)	450
(3,500)	2300

Q6(a) Page fault and OS actions

Question: Under what circumstances do page faults occur? Describe the actions taken by the operating system when a page fault occurs.

Solution:

- A page fault occurs when the CPU references a page whose valid page-table entry indicates that the page is not currently in main memory.
- The hardware raises a trap and transfers control to the OS page-fault handler.
- The OS first checks whether the reference is legal. If illegal, the process is terminated or signaled.
- If the reference is legal, the OS locates the required page on secondary storage.
- The OS finds a free frame or selects a victim page using a replacement policy.
- If the victim page is dirty, it is written back to disk.
- The required page is loaded into the selected frame, the page table and TLB are updated, and the faulting instruction is restarted.

Page-fault handling flow

```

CPU references page
|
v
Is page in memory? -- yes --> continue execution
|
no
|
v
Trap to OS --> validate address --> get free frame / replace victim --> read page from disk
|
v
Update page table/TLB --> restart faulting instruction

```

Q6(b) LRU page replacement

Question: How many page faults occur for the LRU page replacement algorithm for the following reference string with four page frames? 1, 2, 1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 9, 7, 8, 9, 5, 1

Solution:

- Simulating LRU with four frames over the complete reference string gives **12 page faults**.
- The first references fill the frames; later faults replace the least recently used pages at each miss.

Q6(c) Short notes

Question: Write short notes on (i) copy-on-write, (ii) thrashing, (iii) global vs local allocation, (iv) Belady's anomaly, and (v) demand paging.

Solution:

- **Copy-on-write:** parent and child initially share the same physical pages after fork; an actual copy is created only when one of them writes to a shared page.
- **Thrashing:** the system spends most of its time paging instead of executing useful instructions, usually because the working sets do not fit in memory.
- **Global vs local allocation:** in global allocation a process may replace frames belonging to other processes; in local allocation it can replace only its own frames.
- **Belady's anomaly:** in some algorithms such as FIFO, increasing the number of frames can unexpectedly increase the number of page faults.
- **Demand paging:** pages are loaded into memory only when they are first referenced.

Q7(a) Paging example for a 32-byte memory with 4-byte pages

Question: Using the given page table for the 32-byte memory with 4-byte pages, map logical addresses 13, 3, 8, 6, and 15 to physical addresses. Show calculations.

Solution:

- Page size = 4 bytes, so **page number = logical address // 4** and **offset = logical address mod 4**.
- From the readable page table in the scan, use the mapping **page 0 -> frame 5, page 1 -> frame 6, page 2 -> frame 1, page 3 -> frame 2**.
- Logical 13: page 3, offset 1 -> frame 2 -> physical = $2 \times 4 + 1 = 9$.
- Logical 3: page 0, offset 3 -> frame 5 -> physical = $5 \times 4 + 3 = 23$.
- Logical 8: page 2, offset 0 -> frame 1 -> physical = $1 \times 4 + 0 = 4$.
- Logical 6: page 1, offset 2 -> frame 6 -> physical = $6 \times 4 + 2 = 26$.
- Logical 15: page 3, offset 3 -> frame 2 -> physical = $2 \times 4 + 3 = 11$.

Paging translation sketch

```

Logical address --> split into (page, offset)
      |
      v
    page table
      |
      v
frame number + offset --> physical address

```

Q7(b) Compaction and memory organization

Question: Answer the following: (i) Define compaction and explain why it is used. (ii) Compare contiguous memory allocation and paging with respect to external fragmentation and internal fragmentation.

Solution:

- **Compaction** is the process of moving allocated memory blocks so that scattered free holes combine into one large contiguous free area.
- It is used to overcome **external fragmentation** in contiguous allocation schemes.
- In **contiguous allocation**, external fragmentation is a major problem because free memory becomes split into holes of different sizes.
- In **paging**, external fragmentation does not occur because any page can be placed in any free frame.
- However, paging may suffer from **internal fragmentation** because the last page of a process may not be completely full.

Q7(c) Effective memory access time (EMAT)

Question: A demand-paging system has memory access time 100 ns, page fault service time 8 ms, and page fault rate 0.1%. Calculate the effective memory access time.

Solution:

- Convert the page fault service time to nanoseconds: $8 \text{ ms} = 8,000,000 \text{ ns}$.
- Page fault rate $p = 0.1\% = 0.001$.
- $\text{EMAT} = (1 - p) \times 100 + p \times (8,000,000 + 100)$.

- $EMAT = 0.999 \times 100 + 0.001 \times 8,000,100 = 99.9 + 8000.1 = \mathbf{8100 \text{ ns}}$ approximately.
- Therefore, the effective memory access time is about **8.1 microseconds**.

Q8(a) Semaphore synchronization

Question: Processes P1, P2, and P3 must satisfy the order S1 before S2 and S2 before S3. Ensure synchronization among the processes using semaphores.

Solution:

- Use two semaphores: **sem12** for controlling S1 → S2 and **sem23** for controlling S2 → S3.
- Initialize both semaphores to 0.
- P1 executes S1 and then signals sem12.
- P2 waits on sem12 before executing S2, and then signals sem23.
- P3 waits on sem23 before executing S3.

Semaphore-ordering sketch

Initial values: sem12 = 0, sem23 = 0

```
P1: S1; signal(sem12)
P2: wait(sem12); S2; signal(sem23)
P3: wait(sem23); S3
```

Guaranteed order: S1 → S2 → S3

Q8(b) Multithreading models, microkernel, concurrency vs parallelism, dual-mode operation

Question: Briefly explain (i) many-to-many, one-to-one, and many-to-one multithreading models, (ii) microkernel approach of OS design, (iii) concurrency vs parallelism, and (iv) dual-mode operation in OS.

Solution:

- **Many-to-one:** many user threads are mapped to one kernel thread; this is lightweight but cannot exploit multiple CPUs in parallel.
- **One-to-one:** each user thread maps to one kernel thread; this supports true parallelism but has higher overhead.
- **Many-to-many:** many user threads are multiplexed over a set of kernel threads; this balances flexibility and scalability.
- **Microkernel:** only the most essential OS functions remain in kernel mode; services such as drivers and file systems can run in user space.
- **Concurrency vs parallelism:** concurrency means multiple tasks make progress during overlapping time intervals; parallelism means tasks literally execute at the same time on different cores/CPU's.
- **Dual-mode operation:** the CPU operates in user mode and kernel mode so that privileged instructions execute only under OS control.

2021 Exam - Section B

Q5(a) Why page size is always a power of 2

Question: Why are page sizes usually chosen as a power of 2?

Solution:

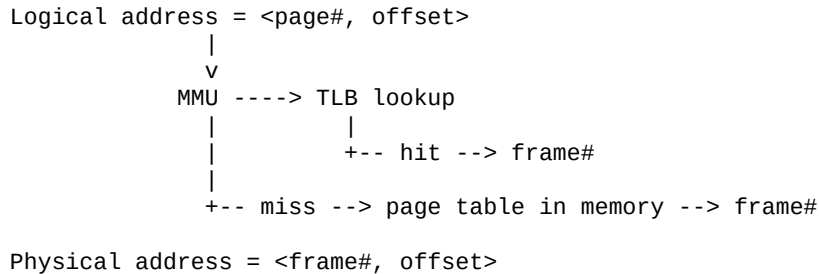
- A power-of-2 page size allows the address to be divided cleanly into a page-number field and an offset field.
- The offset can then be extracted directly using bit operations instead of costly arithmetic.
- This makes hardware implementation simpler and faster.

Q5(b) Logical-to-physical address translation in paging

Question: With a neat sketch, explain how a logical address is translated into a physical address using the paging mechanism.

Solution:

- The CPU generates a logical address consisting of a page number and an offset.
- The MMU uses the page number to index the page table and obtain the frame number.
- The frame number is then combined with the offset to form the physical address.
- If a TLB is present, the MMU first checks the TLB to avoid an extra memory access.

Address-translation sketch**Q5(c) First-fit, Best-fit, and Worst-fit allocation**

Question: Given memory partitions 500K, 100K, 200K, 300K, and 600K (in order), show how First-fit, Best-fit, and Worst-fit place processes of size 418K, 202K, 5K, 112K, and 96K (in order). Which algorithm makes the most efficient use of memory?

Solution:

- **First-fit:** 418 -> 500, 202 -> 300, 5 -> remaining 82 of the first block that fits, 112 -> 200, 96 -> 100.
- **Best-fit:** 418 -> 500, 202 -> 300, 5 -> smallest adequate remainder block, 112 -> 200, 96 -> 98-sized remainder; this tends to minimize immediate leftover space.
- **Worst-fit:** 418 -> 600, 202 -> 500, 5 -> largest remaining block, 112 -> 298 remainder, 96 -> 295 remainder.
- For this data, **Best-fit** is usually considered the most efficient because it attempts to leave the smallest adequate remainder each time.

Algorithm	Allocation order summary
First-fit	418->500, 202->300, 5->82-rem, 112->200, 96->100
Best-fit	418->500, 202->300, 5->82-rem, 112->200, 96->98-rem
Worst-fit	418->600, 202->500, 5->300, 112->298-rem, 96->295-rem

Q5(d) Segment-table physical address problem

Question: Using the given segment table, find the physical addresses for the listed logical addresses.

Solution:

- The general method is always the same: check whether the offset is less than the segment length.
- If valid, physical address = base + offset.
- If invalid, the system raises a segmentation fault / addressing error.

Note: The numeric values of the scanned table are not fully legible in the supplied PDF image, so the reliable complete answer is the translation method.

Q6(a) What happens during a page fault

Question: What happens if a process tries to access a page that was not brought into memory? Explain the steps in handling a page fault with a relevant figure.

Solution:

- The reference generates a page-fault trap because the page-table entry indicates that the page is absent.
- The OS validates the address, locates the page on backing store, allocates or reclaims a frame, and loads the page.

- It updates the page table/TLB and restarts the interrupted instruction.

Page-fault sketch

Reference page --> not present --> page-fault trap --> OS handler --> disk read --> update tables -->

Q6(b) Page faults for LRU and Optimal

Question: For the reference string 1,2,3,4,5,1,2,3,4,1,2,3,5,1,2,3 with four page frames, how many page faults occur for (i) LRU and (ii) Optimal page replacement?

Solution:

- Simulating LRU gives **10 page faults**.
- Simulating the Optimal algorithm gives **7 page faults**.
- Optimal performs better because it replaces the page whose next use is farthest in the future.

Algorithm	Page faults
LRU	10
Optimal	7

Q6(c) Inverted page table and hashed page table

Question: What is an inverted page table? Briefly explain the structure of a hashed page table.

Solution:

- An **inverted page table** has one entry per physical frame, storing the process identifier and virtual page number currently mapped to that frame.
- A **hashed page table** uses a hash function on the virtual page number (often with the process ID) to locate the corresponding page-table entry quickly.
- Hashed page tables are especially useful for very large virtual address spaces.

Q7(a) Memory reference time with and without TLB

Question: Consider a paging system with the page table stored in memory. (i) If one memory reference takes 50 ns, how long does a paged memory reference take? (ii) If a TLB is added and 75% of all page-table references are found in the TLB, what is the effective memory reference time? Assume the TLB lookup time is 0.000002 ms when the entry is present.

Solution:

- **(i)** Without a TLB, one memory access is needed for the page table and one more for the actual data. So total time = 50 ns + 50 ns = **100 ns**.
- **(ii)** Convert the TLB lookup time: 0.000002 ms = 2 ns.
- With TLB hit ratio $h = 0.75$, the effective access time is: $EAT = h \times (2 + 50) + (1 - h) \times (2 + 100)$.
- $EAT = 0.75 \times 52 + 0.25 \times 102 = 39 + 25.5 = \mathbf{64.5 \text{ ns}}$.

Q7(b) RAID 0, RAID 1, and RAID 5 comparison

Question: Compare RAID 0, RAID 1, and RAID 5 in terms of performance improvement, ability to withstand failure, and space overhead required to implement the RAID level.

Solution:

- **RAID 0:** best raw performance because of striping, but no failure tolerance; any single disk failure loses the array.
- **RAID 1:** strong failure tolerance due to mirroring, but requires the highest storage overhead.
- **RAID 5:** good read performance and single-disk fault tolerance with moderate parity overhead.

Level	Performance	Failure tolerance	Space overhead
RAID 0	High	None	Lowest

RAID 1	Good reads, safe writes	High	Highest
RAID 5	Good overall	Survives one disk failure	Moderate

Q7(c) Disk scheduling with cylinders 0-199

Question: On a disk with 200 cylinders numbered 0 to 199, the last head position is 100 and the queue is 55, 57, 37, 15, 80, 160, 150, 36, 185. Find the total head movement for FCFS, SSTF, C-SCAN, and LOOK.

Solution:

- **FCFS:** total movement = **507**.
- **SSTF:** total movement = **255**.
- **C-SCAN** (initial direction upward): total movement = **378**.
- **LOOK** (initial direction upward): total movement = **255**.

Algorithm	Total head movement
FCFS	507
SSTF	255
C-SCAN	378
LOOK	255

Q8(a) fork() and thread_create()

Question: Given the code with one initial fork(), an extra fork() and thread_create() inside the child branch, and a final fork() afterward, find (i) how many unique processes are created, and (ii) how many unique threads are created.

Solution:

- Start with one process containing one thread.
- The first **fork()** creates one child, so there are now 2 processes.
- Only the child enters the if-branch. Inside that branch, another **fork()** creates one more process, making 3 processes in total at that point.
- The child branch also executes **thread_create()**, producing one extra thread in that branch lineage.
- After the branch, every existing process executes the final **fork()**, doubling the process count from 3 to **6**.
- Counting threads across the final processes gives **8 total threads**.

Quantity	Answer
Unique processes	6
Total threads after all forks and thread creation	8

Q8(b) Concurrency without parallelism

Question: Is it possible to have concurrency but not parallelism? Explain.

Solution:

- Yes. On a single-core CPU, multiple tasks can make progress in overlapping time intervals through context switching and time slicing.
- This is concurrency because several tasks are in progress, but it is not parallelism because only one task executes at any instant.

Q8(c) Discrimination in favor of short processes

Question: Explain how much the following scheduling algorithms discriminate in favor of short processes: (i) FCFS, (ii) RR, (iii) Multilevel feedback queues.

Solution:

- **FCFS:** weak preference for short jobs. A short job can wait a long time behind a long job (convoy effect).
- **RR:** moderate preference, especially when the time quantum is small; short interactive jobs often finish quickly.
- **Multilevel feedback queue:** strongest preference among the three because short and interactive processes stay at higher-priority queues and receive faster service.

End of document.